

1 Concepts, Principles & Terminology

Next: [2 Bedrohungen, Angreifer, CVSS, STRIDE](#)

IT Security:

Schutz von Informationen, die in IT-Systemen gespeichert und verarbeitet werden können.

Maßnahmen: Zugangskontrollen, Verschlüsselung, Netzwerksicherheit (Firewalls, VPNs), Backup- und Notfallkonzepte

IT-Sicherheit ist der **Überbegriff**: Alles, was die Sicherheit von Informationen und IT-Systemen betrifft (intern + extern)

Cybersecurity:

Schutz von Informationen, die in IT-Systemen gespeichert und verarbeitet werden können, **die mit dem Internet oder ähnlichen Netzwerken verbunden sind**.

Cybersecurity ist ein **Teilbereich der IT-Sicherheit**, der sich speziell auf Bedrohungen aus dem Cyberspace/Internet konzentriert

Maßnahmen: Penetration Tests, Schutz vor Malware und Phishing

Information Security:

Schutz von **Informationen**, die auf Papier und in IT- und Kommunikationssystemen gespeichert und verarbeitet werden können.

Sensitive Information Classification

Personally identifiable information (PII):

Jegliche Informationen die zur eindeutigen Identifikation von Individuen verwenden werden können

Protected Health Information (PHI):

Untergruppe von PII - Geschützte Gesundheitsdaten

CIA Triad

Confidentiality (Vertraulichkeit):

Die Praxis, **Geheimnisse zu bewahren**, die Privatsphäre zu wahren oder Wertsachen zu verstecken

- Verschlüsselung
- Autorisierung

Integrity:

dass die Daten **nicht** in einer unbefugten Weise oder durch unbefugte Personen **verändert wurden**

- Autorisierung
- Prüfsummen
- Signaturen

Availability (Verfügbarkeit):

Informationen, Ressourcen oder Dienstleistungen müssen verfügbar sein, wenn berechtigte Parteien sie benötigen

- Load-Balancing
- Redundanz

Principles

- Layered Defense

- mehrere Schichten an Verteidigungen die gegen verschiedene Angriffe schützen
- Least Privilege/ Need-to-know
 - nur Erforderliche Ressourcen zu Verfügung stellen -> Minimierung von Schäden
- Separation/ Rotation of Duty
 - keine einzelne Person kann eine schädliche Handlung ohne die Beteiligung anderer durchführen
- Defense in Depth
 - Kombination aus Sicherheitsmaßnahmen auf verschiedenen Ebenen (Netzwerk, Anwendung, Daten)
- Path Management
- Access Control
- Strong Authentication
- Encryption
- Auditing and Monitoring
 - Protokolle führen und Echtzeitüberwachung implementieren
- Incident Response
 - Plan zur Reaktion auf Vorfälle
- User Education
- Vendor Risk Management
 - Drittanbieter müssen die gleichen Sicherheitsstandards haben wie man selbst
- Data Backup and Recovery
- Secure Development
 - sichere coding practices anwenden um häufige Schwachstellen zu vermeiden
- Network Segmentation
 - Netzwerke in Segmente teilen, mit begrenzter Kommunikation zwischen ihnen
- Continuous Improvement

Other Terminology

Security Policy:

Aussage was erlaubt ist und was nicht

Security Mechanism:

Methode, Werkzeug oder Verfahren zur Durchsetzung einer Security Policy

Weakness/ Flaw:

Zustand in dem ein System anfällig werden kann

Vulnerability (Schwachstelle):

Eine Weakness/ Flaw, durch die die Sicherheitsmechanismen des IT-Systems umgangen werden können

Threat (Bedrohung):

mögliche Verletzung eines Sicherheitsziels

Risk:

Wahrscheinlichkeit eines Schadens und Schadenshöhe

Exploit

Eine Methode die Sicherheitsrichtlinien mittels einer Sicherheitslücke verletzt

Attacks:

Versuch ein Sicherheitsziel zu verletzen

Attacker:

obvious

Authentication:

Überprüfen, ob eine Person wirklich diese Person ist

Authorization:

Überprüfen, ob eine Person auf eine Ressource zugreifen darf

Access Rights:

Berechtigungen einer Person für den Zugriff auf eine Ressource

2 Bedrohungen, Angreifer, CVSS, STRIDE

Prev: [1 Concepts, Principles & Terminology](#) Next: [3 Threat Modelling](#)

Security (Sicherheit):

Schutz gegen vorsätzliche Bedrohungen

Safety (Sicherheit):

Schutz gegen zufällige schadhafte Bedrohungen

Klassifizierung von Bedrohungen

Enthüllung (Disclosure) -> unberechtigter Zugang zu Informationen

Täuschung (Deception) -> Entgegennahme falscher Daten

Störung (Disruption) -> Unterbrechung oder Verhinderung der Richtigen Funktion

Übernahme (Usurpation) -> unberechtigte Kontrolle über Ressourcen

Angreifer Modelle

Netzwerk-Angreifer:

Sitzt in der Kommunikationsverbindung zwischen zwei Teilnehmern

- Beobachten des Netzwerkverkehrs
- Unterbrechen "
- Erzeugen von "

Man-in-the-middle: Gezielte Veränderung des Netzwerkverkehrs

Entfernter Angreifer:

Kann mit einem entfernten System über ein Netzwerk interagieren

- Ausführen von Code
- Denial of Service
- Abschöpfen von Informationen

Lokaler Angreifer:

Führt beliebigen Code aus (aber mit beschränkten Zugriffsrechten)

Beispiel: Mehrbenutzer Systeme

Web-Angreifer:

Spezifisch für Web-Anwendungen

Man-in-the-browser: Kann HTTP Anfragen aus dem Browser des Benutzers heraus erzeugen

XSS-Angreifer: Kann JS im clientseitigen Kontext der angegriffenen Anwendung ausführen

Classifying cyber attackers

Amateurs:

little or skill

use existing tools and methods

Hackers:

White hats: with permission and to improve security

Gray hats: compromise systems without permissions

Black hats: for illegal personal, financial or political gain

Cyber criminals:
For financial gain

Hackivist:
Protest against organisations or governments
Leak information

State-sponsored
obvious

Common Vulnerability Scoring System (CVSS)

Common Vulnerabilities and Exposures (CVE) ist eine Schwachstelle die im Rahmen des **Vulnerability Disclosure Prozesses (CVD)** eine Nummer von einer **CVE Numbering Authority (CNA)** erhalten hat, nach dem **Common Vulnerability Scoring System (CVSS)** bewertet wurde und in der CVE-Liste veröffentlicht wurde.

Liste wird von Mitre Corporation verwaltet

Ablauf CVD

1. Sicherheitslücke wird entdeckt
2. CVE Nummer wird mit Beschreibung beantragt
3. Einreichen der Schwäche in die Liste (Keine Veröffentlichung)
4. Weitergabe des CVE an den Hersteller
5. Zeit für Korrektur durch den Hersteller
6. Veröffentlichung des CVE in den offiziellen Listen

Berechnung CVS

- Base Metrics
 - Attack Complexity
 - Privileges Required
- Temporal Metrics
 - Report confidence
- Environmental Metrics
 - Confidential Requirement

STRIDE

Threat	Property	Short definition	Example
Spoofing	Authentication	Impersonating something or someone else.	Pretending to be a valid customer
Tampering	Integrity	Modifying data or code	Modifying a shopping transaction
Repudiation	Non-repudiation	Claiming to have not performed an action	"I did not buy that item"
Information Disclosure	Confidentiality	Exposing information to someone not authorized to see it	Reading information about the transaction (although not authorised)
Denial of Service	Availability	Deny or degrade service to users	Crashing the web site, sending a packet and absorbing seconds of CPU time, or routing packets into a black hole
Elevation of Privilege	Authorization	Gain capabilities without proper authorization	Allowing a remote internet user to run commands on the application server

Datenflussdiagramme

• Prozesse



• Datenquellen / -senken



• Datenfluss

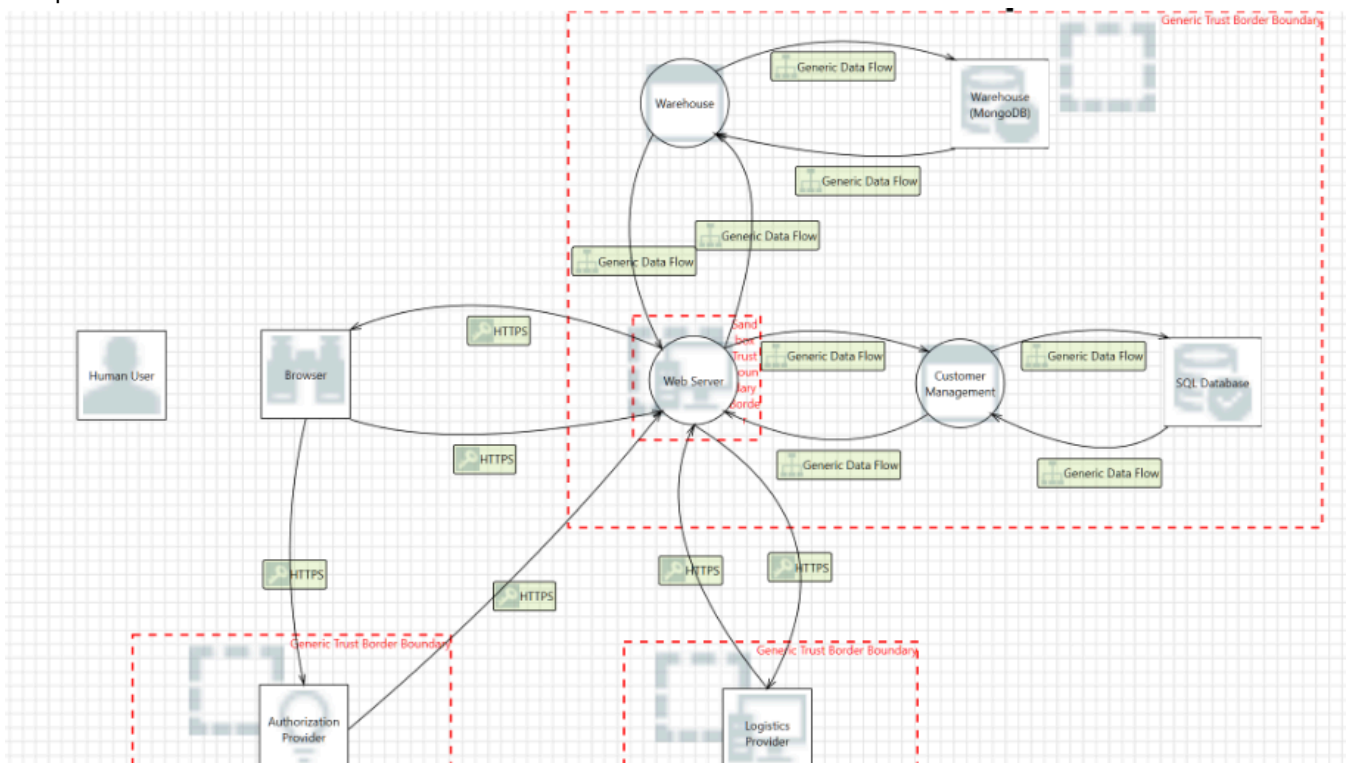


• (Externe) Entitäten



	S	T	R	I	D	E
	X		X			
	X	X	X	X	X	X
		X		X	X	
		X		X	X	

Beispiel:



3 Threat Modelling

Prev: [2 Bedrohungen, Angreifer, CVSS, STRIDE](#) Next: [4 Kryptographie](#)

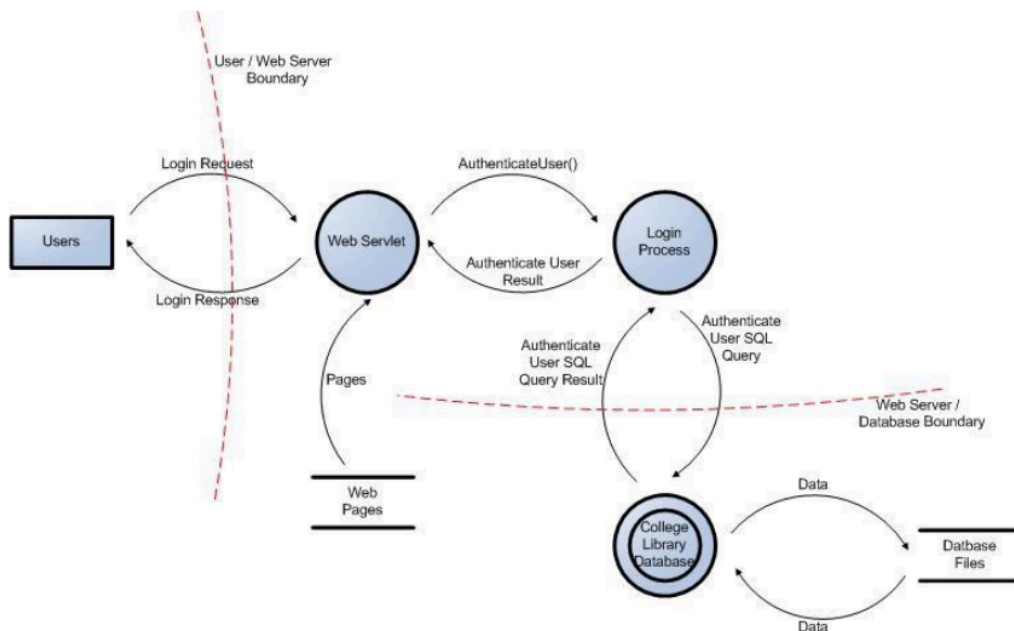
1. Scope your work
2. Determine Threats
3. Determine Countermeasures and Mitigation

TM1: Scope your Work

Thread Model Information Document with the following:

- Version Number
- Description
- Owner and Reviewer

TM1: User Login Data Flow Diagram for the College Library WebSite



Trust Levels:

Access rights that the application will grant to external entities

Assets:

Have something that attackers want

Can be physical (list of clients) or abstract (organization reputation)

Trust Levels are cross-referenced with the entry points and assets -> able to define access rights to each entry point

Entry Points:

Interfaces through which potential attackers can interact with the application or supply it with data

TM2: Determine Threats

Beispiel:

Threat: Account takeover via stolen credentials

Asset: User Login Details

STRIDE category: Spoofing (primary), Information Disclosure, Elevation of Privilege

Description:

An attacker steals or guesses valid user login details (e.g., via phishing or reused passwords) and logs in as a legitimate user or librarian, gaining unauthorized access to data or admin functions.

TM3: Determine Countermeasures and Mitigation

Risiko = Wahrscheinlichkeit * Auswirkung (siehe [1 Concepts, Principles & Terminology](#))

Wahrscheinlichkeit hängt von Einfachheit ab

Auswirkung basiert auf Schadenspotential und Ausweitung

Alternative dazu ist DREAD-D

DREAD-D

Bewertung von Bedrohungen für Software

Jede Kategorie bekommt Wert

Skala ist nicht vordefiniert

Summierung aller Werte

Damage:

Wie groß ist der Schaden bei Angriffsfall - Monetärer Verlust, Datenverlust ...

Reproducibility:

Wie einfach kann der Angriff durchgeführt werden

Exploitability:

Aufwand und Expertise die der Angriff benötigt

Affected Users:

Anzahl der betroffenen Nutzer

Discoverability:

Wie einfach kann diese Lücke gefunden werden

4 Kryptographie

Prev: [3 Threat Modelling](#) Next: [5 Authentication](#)

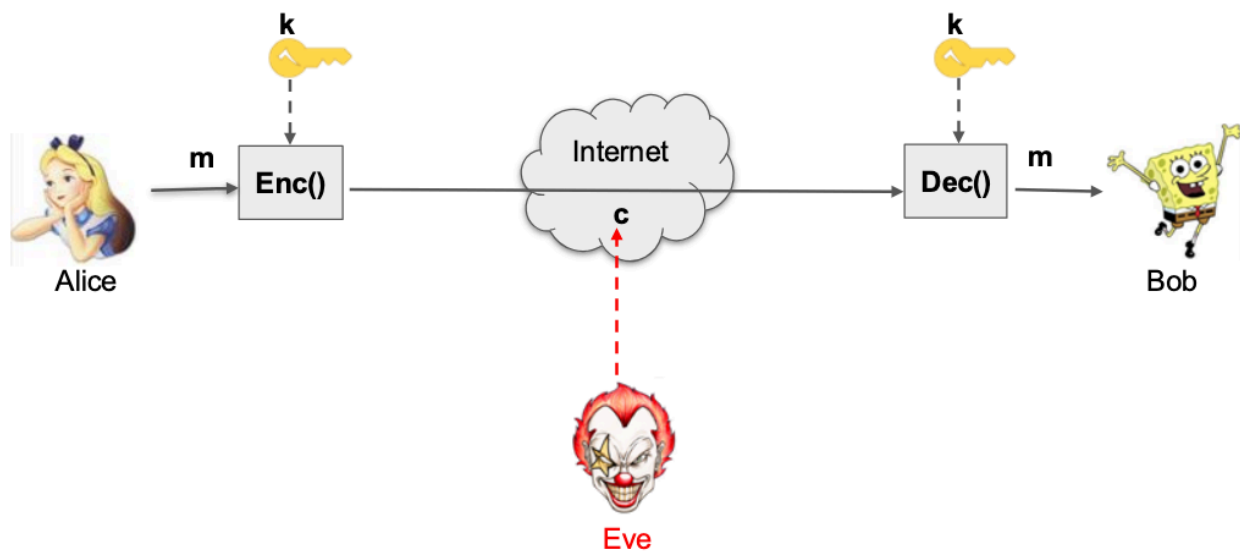
Def: Lehre der Verschlüsselung von Daten.

Kerckhoff's Prinzip:

die Sicherheit eines Kryptosystems darf nicht von der Geheimhaltung des Algorithmus abhängen. Die Sicherheit gründet sich nur auf die Geheimhaltung des Schlüssels

Symmetric Cryptography

Secret/ Private key



Based on a shared secret key

1. Alice encrypts with the key
2. Bob decrypts with the key

Stromchiffren:

verschlüsseln jedes Bit einzeln

Blockchiffren:

verschlüsseln einen Block von Bits gleichzeitig

z.B. Advanced Encryption Standard (AES)

Blocklänge ist immer 16 Byte

Schlüssellänge variiert

Jeder Block einzeln und unabhängig

Problem:

gleiche Blöcke werden gleich verschlüsselt

Modes of Operation

Blöcke zu verknüpfen

Verschlüsselung nicht deterministisch zu machen

Cipher Block Chaining mode (CBC):

1. Create an initialized vecotor (IV)
A block of the same size as the rest of the blocks, containing random values
2. For each Block:

```

if first block:
    cipher[0] = Encrypt(block[0] XOR IV)
else:
    cipher[x] = Encrypt(block[x] XOR block[x-1])

```

Decrypting:

```

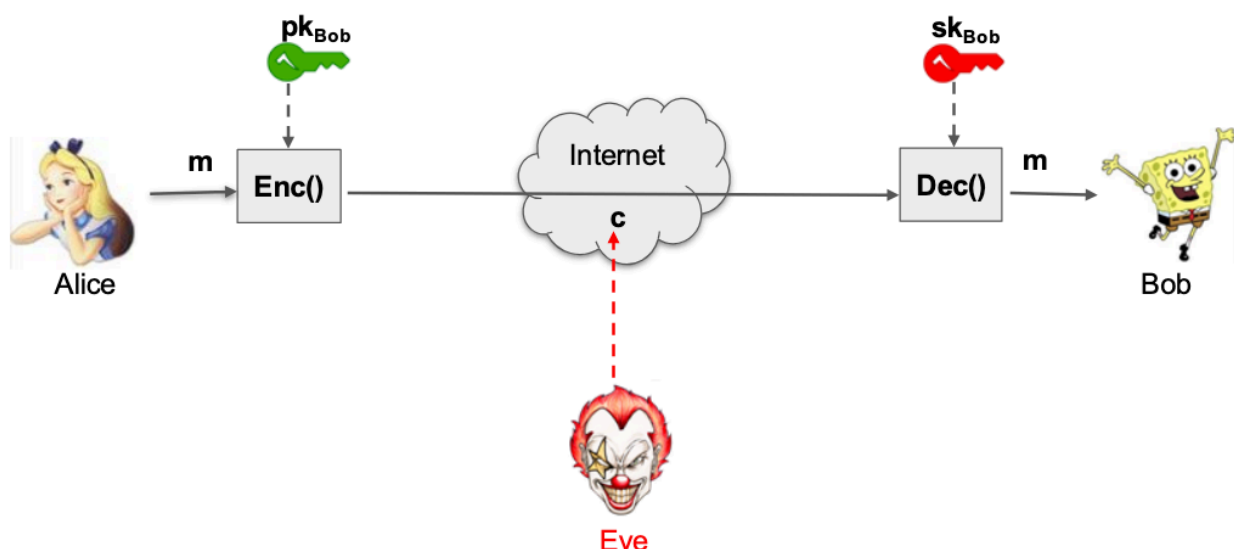
block = IV + block
plaintext = Decrypt(block[x]) XOR block[x-1]

```

IV is not secret, it is sent with the ciphertext

Asymmetric Cryptography

Everyone can send a message with the public key but only the owner of the private key can decrypt the message



Schlüssellängen > 2048 Bit

RSA

Algorithmus:

```

p, q = (big) prime numbers
N = p * q
c = (p - 1)*(q-1)
e = so that 0 < e < c and ggT(e, c) = 1
d = so that d * e \equiv 1 mod c

```

Private key: (N, d)

Public key: (N, e)

Zwei Primzahlen zu multiplizieren ist einfach, die Umkehrung ist extrem schwer.

Verschlüsselungsfunktion:

$$c \equiv m^e \pmod{N}$$

Entschlüsselung:

$$m \equiv c^d \pmod{N}$$

Elliptic Curve Cryptography (ECC)

Steht nicht auf seiner Klausurthemenübersicht

Vorteile/ Nachteile von (A)symmetrischer Verschlüsselung

	Symmetrisch	Asymmetrisch
Vorteile	- Weniger Rechenintensiv als Public Key Krypto (Asymmetrisch??) - Performance - Optimierung für Hardwareimplementierung möglich - kürzere Schlüssel	- Publik Key kann öffentlich abgelegt werden
Nachteile	Schlüsselverteilungsproblem (Jeder Teilnehmer muss den Schlüssel kennen)	- lange Schlüssel - Rechenintensiv - Nachricht darf max. so lang sein wie der Schlüssel ()

Hybride Kryptoverfahren

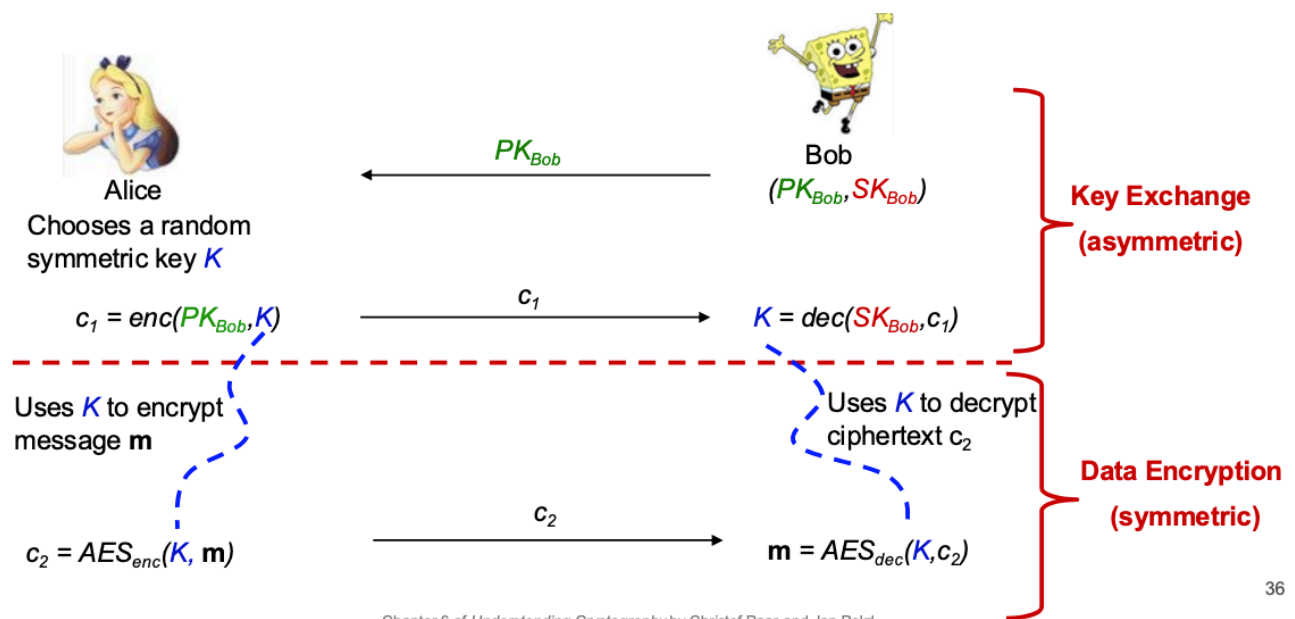
Public Key Krypto um die symmetrischen Schlüssel zu verteilen

-> Löst Schlüsselverteilungsproblem

Daten mit symmetrischer Krypto verschlüsseln

-> Performance Gewinn

-> beliebig lange Daten können verschlüsselt werden



36

Kryptographische Hash Funktionen

Bilden beliebigen Input auf einen Output mit fixer Länge ab

- Einwegfunktion
- Starke Kollisionsresistenz

Message Authentication Codes (MAC)

Temper proof signature so you can check:

- Who sent it? (authenticity)
- Whether it was changed? (integrity)

Uses a shared secret key -> without key the hash can not be verified

```

M = message
K = secret key
MAC() = MAC algorithm
# sender computes a tag:
tag = MAC(K, M)
# Sender sends:
M, tag
# Receiver computes:
tag2 = MAC(K, M)
if tag == tag2:
    # Message is authentic

```

The message is not encrypted by this, this has to be done separately

HMAC is a standard algorithm that uses a hash function like SHA

Signaturen

Basiert of public keys

Beim Signieren aber umgedreht

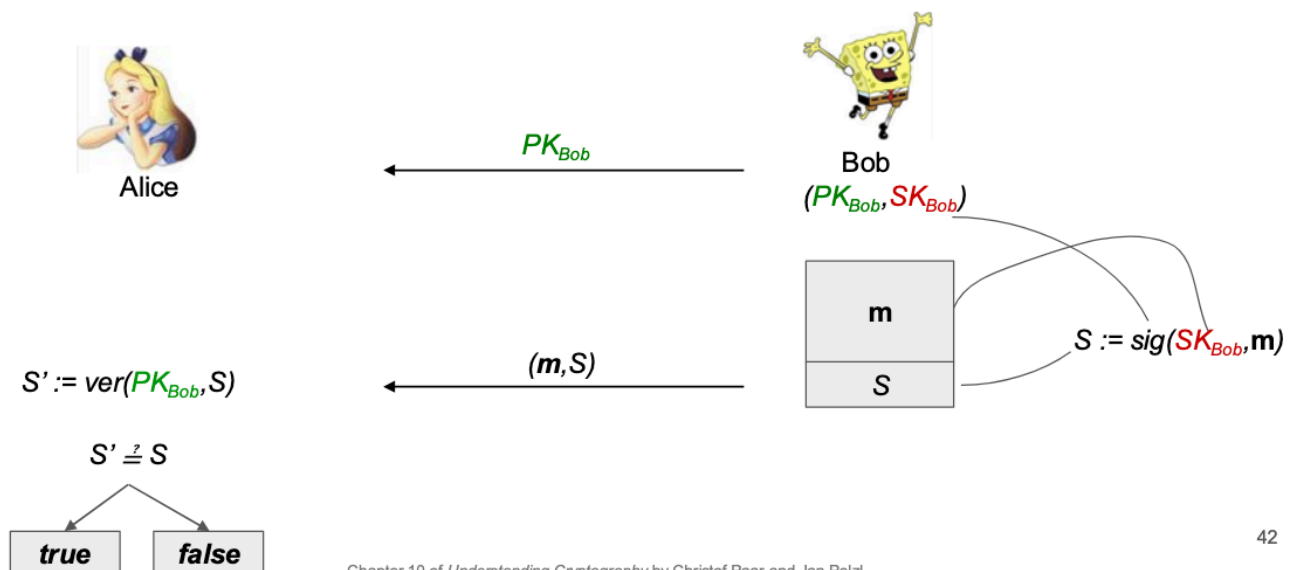
Ziel: Eindeutige Autoren

Signieren mit Private Key

Verifizieren mit Public Key

Hash der Nachricht wird signiert

RSA, DSA, ECDSA



5 Authentication

Prev: [4 Kryptographie](#) Next: [6 Malware](#)

Authentication vs Authorization see [1 Concepts, Principles & Terminology](#)

Authentication Goals

- No false negatives (No falsely denying access)
- No false positives (No falsely allowing access)

Authentifizierungsfaktoren

- something you know
- something you have
- something you are

Passwords

not a strong defence on its own, because:

- people use weak passwords
- Human memory limitations

Weak password:

easy to guess/ predict

Characteristics:

- short
- Use a common word
- predictable sequence of characters
- personal information in a password

Threats against Password Authentication

- offline cracking
- keylogging
- social engineering

Replay Attack:

Attacker sniffs the hash from a true message and blocks it from reaching the recipient

Attacker can then replay the message at a later time

For example password when he needs it

Brute Force Attack:

- offline
- use program to automatically try every possible combination
- can be reduced with parameters
 - 62⁶ possible passwords takes about 10 min (upper/ lower case + digits)
- use rainbow table to reduce search time (common words and passwords)
- users use upper case letters at the beginning and numbers at the end

Salting

add a random salt to the password before hashing -> defeats rainbow tables

store the salt in plain text with the password hash

Why plain text?

Salts are not secrets -> does not help to reverse the hash

salts force the attackers to use a new rainbow table for every salt/ user -> hashes differ between users even if passwords are the same

-> choose a relatively slow hash function so that cracking becomes more difficult

Peppering

add a random pepper to the password before hashing

store the pepper in another physically/ virtually separated location

Linux password security

previously stored as hash in `/etc/passwd`, which was publicly available to all users, only superuser could change

now stored in `/etc/shadow` only visible to the superuser

Something you are

Examples:

- Fingerprint
- Retina scans
- Keystroke dynamics

Advantages:

- cannot be disclosed, lost, forgotten

Disadvantages:

- cost for installation and maintenance
- reliability
- can possibly be forged

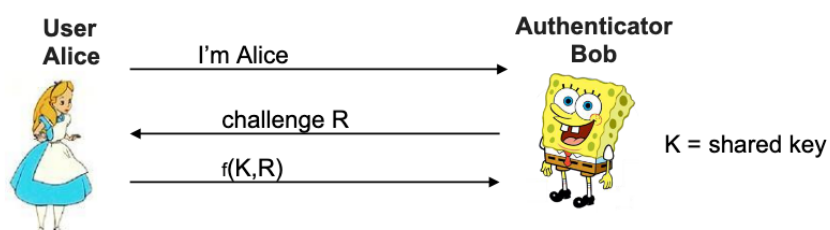
Something you have

Examples:

- Tokens, hardware keys

Challenge Response

Simple Challenge-Response

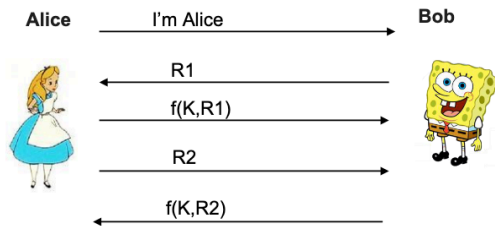


Prevents replay attacks is used only once and is random

Problem:

- Bob could malicious

Two-way Challenge-Response



prevents MiM attacks, as both sides authenticate

Problem:

- Vulnerable against reflection attacks, if initiator does not respond before target

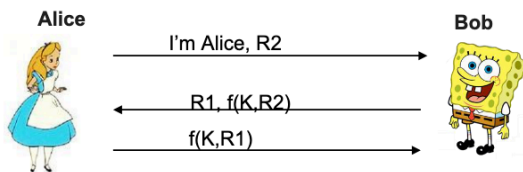
Reflection Attack:

when an attacker takes a challenge meant for them and **reflects it back** to the legitimate party so the legitimate party ends up solving its own challenge

how this effects two-way challenge response:

- **Alice** → **Eve**: "Challenge: NA"
- **Eve** → **Alice**: "Challenge: NA" (reflected)
- **Alice** → **Eve**: "Response: $f(NA, K)$ "
- **Eve** → **Alice**: "Response: $f(NA, K)$ " (reflected back)

more efficient two-way challenge response:



One time Passwords (OTP)

- not vulnerable to replay attacks
- avoids the problem of same password on many systems

Challenges:

- Time synchronisation
- Delivery of OTP

Time-Synchronised OTP Generation

- Token clock is synchronised with authentication server
- generate new OTP based on current time

OTP with Security Token

- generates a random number that changes every 60 sec
- Seed shared with the server

new OTPs are created from the past OTP

Lamport's OTP algorithm

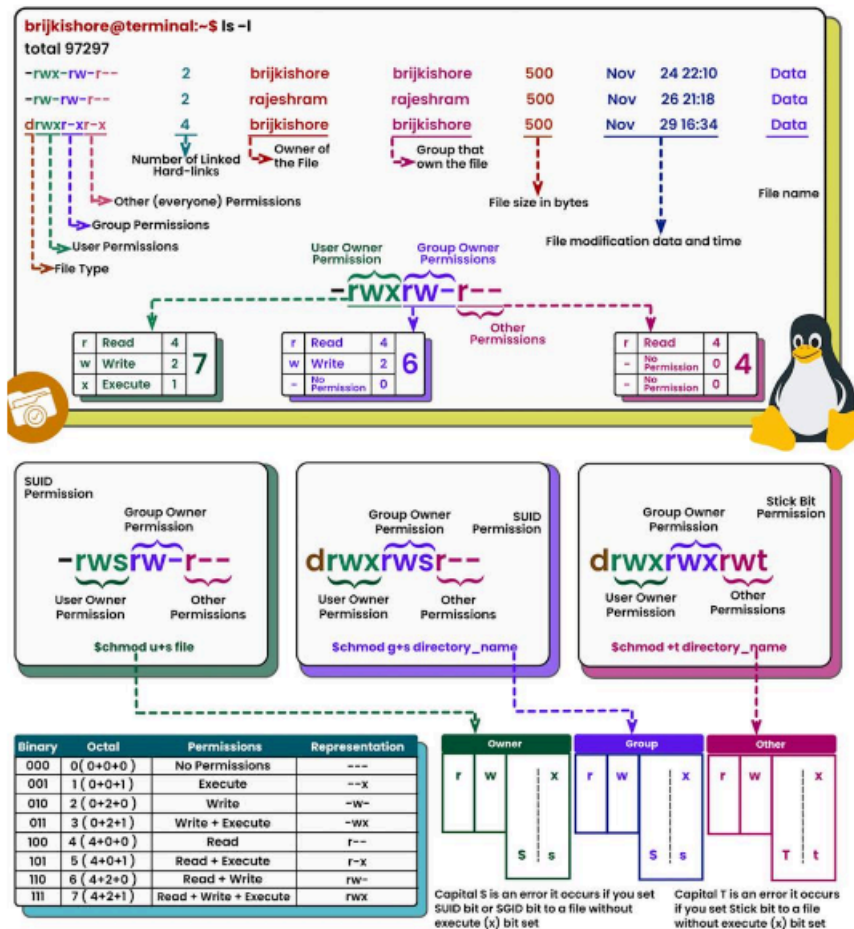
Multifactor Authentication (MFA)

- must include unique factors
- Attackers must defeat both to compromise the authentication

Authentication over a Network

- Public Key Interfaces (PKI)

Linux File Permission Cheatsheet



Discretionary Access Control (DAC)

The owner of the Ressource controls how has which permissions on it

Problem:

- If a user has access to data, they can often **copy it, re-share it, or forward it** to others.
- The system can't stop the data from flowing to people who shouldn't have it.
- DAC assumes users will: behave responsibly
- In real organisations: Employees don't decide who gets access to HR data.

Mandatory Access Control (MAC)

- Access level is based on security clearance levels
- Every object gets a classification (Public, confidential, top secret...)
- Admin grants rights according to access level

6 Malware

Prev: [5 Authentication](#) Next: [7 Network Security](#)

Malware:

Software mit bösartiger Wirkung

Malware Einordnung

- Replikation (self-replicating): Malware versucht sich aktiv zu verbreiten (herstellen von Kopien von sich selbst)
- Populationswachstum (population growth): Veränderung der Anzahl der Anzahl der Malware-Instanzen -> nur well self-replicating
- Parasitismus (parasitic): Malware benötigt Wirt um zu existieren (Source Code, Makros...)

Maleware Typen

Logische Bombe (logic bomb)

Besteht aus Nutzlast (böswillige Aktivität) und einem Auslöser (Boolesche Bedingung, die entscheidet ob die Nutzlast ausgeführt wird)

Ist klein und deswegen schwer im Sourecode auffindbar

Replikation: Nein

Populationswachstum: -

Parasitismus: Möglich

Trojanisches Pferd (trojan horse)

Program das eine vom Nutzer gewünschte und eine ungewünschte Funktionalität enthält

Replikation: Nein

Populationswachstum: -

Parasitismus: Ja

Hintertür (back door)

dediziert dafür vorgesehen, Sicherheitsmaßnahmen des Systems zu umgehen, auf dem sie existiert
Hintertüren können in Programme eingefügt sein (Trojanisches Pferd) oder eigenständig existieren

Replikation: Nein

Populationswachstum: -

Parasitismus: Möglich

Wurm (worm)

verbreitet sich beim Ausführen, indem er sich über ein Netzwerk im Wirtssystemen einnistet.

Ein Wurm existiert eigenständig im Wirtssystem.

Replikation: Ja

Populationswachstum: Positiv

Parasitismus: Nein ?? #QEST

Virus (virus)

Malware die sich beim ausführen verbreitet, indem sie sich sin neue Wirte einnistet (Code in ausführbaren Binärcode einführen)

Dropper: Trojanisches Pferd das den Virus startet

Replikation: Ja
Populationswachstum: Positiv
Parasitismus: Ja

Kaninchen/ Fork Bomb (rabbit/ fork bomb)

möglichst schnelle Replikation um das Wirtssystem zu erschöpfen

Replikation: Ja
Populationswachstum: Positiv
Parasitismus: Nein

Spyware

vertrauliche Daten über das Netzwerk an verborgenen Empfänger sendet

Replikation: Nein
Populationswachstum: -
Parasitismus: Nein

Adware

zeigt Werbung an
ermittelt das Konsumprofil des Nutzers
und versendet dieses über das Netzwerk
(Subklasse der Spyware)

Replikation: Nein
Populationswachstum: -
Parasitismus: Nein

Zombie und Botnetze

Stellt Angreifern bösartige Funktionalität über des Netzwerk zur Verfügung stellt
Ein Verbund von Zombies, der vom Angreifer wie eine Einheit ferngesteuert werden kann, nennt man Botnets.

Replikation: Nein
Populationswachstum: -
Parasitismus: Nein

Viren

```
def virus():  
    infect()  
    if trigger() is true:  
        payload()  
  
def infect():  
    repeat k times:  
        target = select_target()  
        if no target:  
            return  
        infect code(target)
```

Beispiel Targets:

- Boot Sektor
- Ausführbare Datei
- Source-Code-Datei

Infektion des Boot-Sektors

Bootvorgang:

1. Computer anschalten
2. Primärboot: BIOS starten, Selbsttest, Geräteerkennung, Bootgerät identifizieren, Bootblock lesen und ausführen
3. Sekundärboot: Bootblockprogramm lädt Code nach, der Dateisysteme lesen kann, und führt diesen aus
4. Der Betriebssystemkern wird geladen und ausgeführt

Virus kopiert originalen Bootblock und überschreibt ihn

- Virus benötigt zum weiteren Hochfahren des Systems den originalen Bootblock
- Virus erhält Kontrolle beim Bootvorgang noch bevor Betriebssystem und Anti-Viren-SW geladen werden

Gegenmaßnahme: Blockschutz im BIOS

Infektion von Dateien

- Zeildateien: ausführbare Dateien, Dokumente mit interpretierbarem Programmcode
- Virus fügt sich in Dateien ein
- Virus erhält Kontrolle beim Ausführen der Datei

Dateianfang überschrieben:

- Dateigröße unverändert
- Wirtsprogramm zerstört
In der Mitte überschreiben:
- Dateigröße unverändert
- Wirtsprogramm läuft anfangs, ist dennoch defekt
Vollständig ersetzen:
- Leicht entdeckbar durch identische Dateigrößen der verschiedenen Vireninstanzen

Es gibt auch Viren die sich an den Code anhängen. (Im Header neuer Einstieg in die Datei, bei Binärdateien häufigster Fall)

Nachteil:

- Einfach zu erkennen, da Dateigröße verändert (Wird von Virensclannern standardmäßig überprüft)
Vorteil:
- Weniger offensichtlich für den Nutzer

Einsteuern in ungenutzte Füllbereiche

- Dateigröße unverändert
- Wirtsprogramm läuft korrekt

Zusatz:

- **Analyse der Datei**
Das Malware-Programm sucht innerhalb des Headers oder der Sektionen nach ungenutztem Platz.
- **Code-Split**
Schadcode wird in kleine Fragmente zerlegt, die in die gefundenen Hohlräume passen.
- **Einfügen**
Diese Fragmente werden in die Lücken verteilt, ohne die Datei zu vergrößern.
- **Manipulation des Control Flow**
Ein kleiner Jump/Redirect sorgt dafür, dass die Datei beim Start irgendwann zu den eingefügten Fragmenten springt.
- **Wieder zurück zum Originalprogramm**
Nach Ausführung springt der Code wieder zum legitimen Teil des Programms, damit es scheinbar normal

läuft.

- Seltener weil signifikant schwerer und nicht immer möglich wenn zu wenig cavities existieren

Begleit-Viren:

Virus ist separate File; wir vor dem Wirtsprogramm aufgerufen

- Virus trägt denselben Namen wie der ursprüngliche Wirt -> Wirt wird unbenannt
- Wirts Programm bleibt unverändert

Quellcode-Viren:

Verändert den Quellcode nicht die Byte Dateien

Zusatz:

- Programmierer würden Veränderungen im Quellcode bemerken
- Versionskontrolle (Git) macht Infektionen sichtbar
- Build-Systeme sind heute komplexer
- Sicherheitsüberprüfungen, Code-Reviews und CI/CD verhindern Manipulationen

Dokumenten-Viren (Macro viruses):

Infiziert nicht-ausführbare Dokumente mit Makros

Applikationen, die das Dokumentenformat verstehen, interpretieren die enthaltenen Makros

Passive Erkennungsvermeidung

Vermeiden Erkennung durch Nutzer und Anti-Virus Programme

Spuren Verwischen

Statisch:

- Dateigröße anpassen
- Zeitstempel manipulieren

Dynamisch:

- Ursprüngliche Datei-/ Bootblockmerkmale und -inhalte zur Laufzeit im Speicher dauerhaft vorhalten und bei I/O-Anfragen einspielen

Verschlüsselung: Polymorphie

Verschlüsselter Virus-Body

Einfache Verschlüsselung: kein Schlüssel, einfach umkehrbare Operationen -> Latein

Statische Schlüssel: kurzer Schlüssel, parametrisierte umkehrbare Operationen

Variable Schlüssel: kurzer Schlüssel, wird in jeder Iteration deterministisch verändert, Operation wie in vorher

Substitution: Jedes mögliche Klartextwort wird auf $n \geq 1$ verschiedene Chiffreworte abgebildet

Starke Verschlüsselung: Langer Schlüssel, sicheres Kryptosystem

Code Obfuscation

randomisierte Ersetzung von Code durch andere, äquivalente Codes

Ziel: Reverse Engineering erschweren

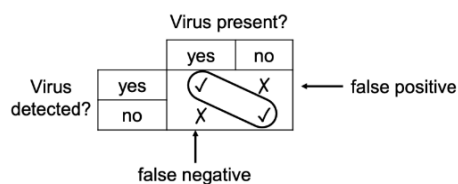
- Äquivalente Befehle
- Äquivalente Befehlssequenzen
 - Befehlssequenzen mit demselben Effekt
- Befehlsumordnung

- Umbenennung
- Datenumordnung
 - Umordnen des Speicherorts von Daten
- Spaghettifizierung:
 - ganz viel `jmp/ condjmp`
- Unnötige Befehle (junk code)
- Code-Erzeugung zur Laufzeit
 - Während der Laufzeit wird der Code verändert
- Nebenläufigkeit
 - Ist einfach verwirrend genug
- Inlining
 - Der Code einer Funktion wird **direkt an den Aufrufort eingefügt**, anstatt über einen Funktionsaufruf ausgeführt zu werden
- Outlining
 - Gegenteil
- Code-Interpretation **#QEST**
- Threaded Outlining
 - Auslagern von Code in Subroutinen ohne Rücksprung

Malware Erkennung

Erkennung:

Erkennen ob Daten einen Virus enthalten



Identifikation:

welcher/ was für ein Virus ist das?

Desinfektion:

entfernen des Virus

Statische Virus-Erkennung durch Scanner

Statische Erkennung:

Virus code muss dafür nicht ausgeführt werden

Virens Scanner:

Überprüft Datenströme ob sie bekannte Signaturen von Malware enthalten

Macht dies auf Anfrage (user aktiviert ihn explizit) oder bei Zugriff (Scanner läuft die ganze Zeit im Hintergrund und Durchsucht beim Zugriff auf die Daten)

Statische Virus-Erkennung durch Heuristiken

Statische Heuristik:

sucht nach Code, der sich "virusähnlich" verhält

Man sucht Anzeichen, dass Code Virus ist oder nicht:

- junk code
- Selbstmodifizierender Code
- Nutzung nicht-dokumentierter API-Aufrufe
- Ungewöhnliche Maschineninstruktionen

Entscheidung:

- aus allen Anzeichen eine Entscheidung berechnen

Vorteil:

- Erkennt bekannte und unbekannte Viren

Nachteil:

- viele falsche Alarme (false positives)
- keine Identifikation
- keine Desinfektion

Verifikation der Datenintegrität

Viren verändern Daten um sich zu verbreiten.

Wenn diese unautorisierte Veränderungen erkannt werden, könnte dies ein Virus sein

Ein Virus hängt sich an ein Programm an -> die Datei wird größer oder ihr Inhalt verändert sich

-> Integritätsprüfung merkt das

Problem:

- Vergleich der Daten mit unifizierten Daten

Ansätze:

1. Periodische Überprüfung (vergleicht Hash von Systemdateien mit den Referenzwerten)
2. Programm überprüft beim Programmstart ob es nur unverändert ist
3. Prüfung durch Betriebssystem (z.B. nur signierte Programme werden ausgeführt)

Vorteile:

- hohe Performanz
- erkennt bekannte und unbekannte Viren

Nachteile:

- aufwendige Wartung
- false positives
- keine Identifikation
- keine Desinfektion

Dynamische Virus-Erkennung durch Monitore

Dynamische Erkennung:

Erfordert dass der Code ausgeführt wird

Monitor (behavior monitor/ blocker)

versucht in Echtzeit Virus ähnliches Verhalten zu erkennen

Kann diese im Idealfall blockieren

Missbrauchserkennung (intrusion detection)

Spezifizieren unerlaubter Aktivitäten

Grobkörnige Modellierung:

- Signaturen von Aktivitäten die virusartig sind
- Scanner vergleicht in Echtzeit die Signaturen
- Ignoriert die Parameter der Aktivitäten
- Signaturen kommen aus Expertenwissen

Nachteil:

ist zu ungenau und führt zu false positives

Feinkörnige Modellierung:

- Modellierung von Merkmalen bzw. Parametern der Aktivitäten
- Kombination mit früheren Merkmalen

Vorteil:

- Präzise Identifikation eines Virus

Nachteil:

- Geringe Erkennungsrate bei neuen Viren

Anomalieerkennung (intrusion detection)

Spezifizierung erlaubter Aktivitäten, davon abweichende Aktivitäten ist eine Anomalie

Vorteil:

- Erkennt bekannte und unbekannte Viren

Nachteil:

- Training und Einstellung des Verfahrens für jedes Programm notwendig
- Keine Identifikation
- Keine Desinfektion

Emulation

Dynamische Heuristiken:

wie statische

nur zur Laufzeit gelieferte Beobachtungsdaten

Generische Entschlüsselung:

Virus entschlüsselt sich zur Laufzeit

#TODO hä

Vorteil:

- Virus befällt nur emulierte Umgebung
- Echtsystem bleibt sauber
- Erkennt bekannte und unbekannte sowie polymorphe Viren

Nachteil:

- hoher Ressourcenaufwand
- unerkennbare Emulation schwierig zu implementieren
- Virus kann während der Emulation inaktiv bleiben
- keine Identifikation
- keine Desinfektion

Aufbau:

- CPU Emulation
- Speicher Emulation
 - Markieren von modifiziertem Speicher
- Betriebssystem-Emulation
- Hardware Emulation

Aktive Erkennungsvermeidung

gezielt gegen Anti-Viren Programme wehren
Methoden die aktive Analyse der Malware zu erschweren

Ziele:

1. Funktionsfähigkeiten unterbinden
2. Virenanalyse erschweren
3. Entdeckung vermeiden indem Funktionsweise der Anti-Viren-SW ausgenutzt wird

Retroviren

Aktives Behindern vorhandener Anti-Viren-SW

- Beenden dieser Prozesse
- Dauerhaftes Abschalten durch Modifikation im Sekundärspeicher
- IP-Adressübersetzungsdatei manipulieren um Aktualisierung zu verhindern

Emulation umgehen, erkennen oder stören

Aussitzen:

- Infektion erst nach Emulationsabbruch durchführen
- unnötige Berechnung ausführen bis Emulator abbricht

Dynamische Heuristiken vermeiden:

- Virus so konzipieren dass diese nicht anschlagen
- Mehrfaches Entschlüsseln des Virus **#TODO** was bedeutet das?

Reverse Engineering verhindern

Virus ist aufgefallen identifiziert

- Debugging Vorgang erkennen und normalen Code ausführen
- Breakpoints des Debuggers erkennen
- Einzelschritt-Modus erkennen
 - Ausführungszeit von Codessequenzen messen
- Disassemblierung erschweren
 - Mischung von Code und Daten
 - Eigene Breakpoint und Interrupt-Handlern

Würmer

Gemeinsamkeiten mit Viren:

- Tarnmechanismen
- Anti-Anti-Viren-SW Methoden
- verschiedene Infektionsvektoren

Unterschiede:

- Würmer benötigen keine Wirtsdateien
- Würmer müssen nicht explizit ausgeführt werden
- Würmer verbreiten sich schneller